



Embedded Systems

Exercise 2:

Queues, Memory Pools, Application Timer, Event Flags

Name:	Group:
Name:	Preparation:
Name:	Hand-in:
Mark:	
Comments:	

Plagiarism:

The students confirm with the submission of the report that all sources used have been attributed and that the report is their own work. Not permissible is the dissemination of their own report or the use of reports from other students.

1. Introduction and Aim

This exercise will deepen the understanding of ThreadX. The exercise is split into four parts.

1. Memory pools – Gatekeeper
2. Application timer
3. Application timer & Semaphore
4. Event flag groups

2. Reporting

This lab exercise does not require a written report. However, the some of the exercises have to be demonstrated during or at the end of the lab session. These exercises are marked with the label **Demo** in the subtitle.

3. Preparation

Each lab exercise requires a preparation. Please look through section 4 and prepare your source code. Information about the ThreadX API can be found in the lecture notes or on github:

<https://github.com/eclipse-threadx/rtoS-docs/blob/main/rtoS-docs/threadx/chapter4.md>

4. Exercise

Information about the board and the programming of it, can be found in the Learning Management System (LMS → Moodle).

Note: It is useful to write these programs in different projects. Some parts of the earlier exercises may be used in later ones.

4.1. Memory pool – Gatekeeper (Demo)

Create a project with three threads. Two threads creating a message string and one thread that acts as a gate keeper for the USART. Use a message queue and a memory byte pool to pass the message strings from thread 1 and 2 to the USART thread.

- Thread 1: creates every 0.8s the following string and sends it to the queue:
"Thread 1: executed X times, current system time: XXX."
- Thread 2: creates every 1.2s the following string and sends it to the queue:
"Thread 2: executed X times, current system time: XXX."
- Thread 3: receives the messages from the queue and sends them to the USART using the HAL library function

4.2. Application timer

Create a new project to implement an application with a periodic application timer. The purpose of the application timer is to toggle LD1 (PI1). Your project needs at least one thread.

- a) Set the count value (reschedule_ticks) to 10 and to 100 and observe the difference. Measure the on and off time using an oscilloscope.
- b) Modify the "Timer Ticks Per Second" property to 1000. Repeat step a).

4.3. Application Timer & Semaphores (Demo)

Create a project with one thread and one application timer. The thread is an LED Thread and toggles LD1 every time it executes. The LED Thread is suspended by waiting for a semaphore. The semaphore is put by the application timer periodically and thus signals the LED Thread to execute.

4.4. Event Flag Groups (Demo)

Create a new project based on 4.1. Modify the programme in the following way:

- Add a second message queue to the programme so that the two threads (1 and 2) have their own message queue.
- Modify the Gate Keeper Thread (USART) in such a way that it uses Event Flags to wait for a message in either of the two message queues (one from thread 1 and one from thread 2). If one of the message queues holds a message, the message needs to be retrieved and send to the USART.

4.5. Performance & Deadline

Import the archived project "AZ_Perf_Deadline" into your workspace (see moodle). The application has three threads (High, Mid, Low) and three application timers. Each thread is triggered by one application timer. Two of the threads use a shared resource which is simulated by a function

"*access_bus()*" and is protected by a semaphore. Important is the timing of the High thread (*Thread_High_entry*). The access to the shared resource needs to happen within 100ms of the activation by the application timer. Thus, the access to the shared resource by the High thread is shown by setting/resetting the port PI1 (D13 on CN7) and the activation is shown by setting/resetting port PI2 (D8 on CN7).

- a) Use TraceX to visualise the scheduling and to understand the application. TraceX is already setup in the project.
- b) Use an oscilloscope to visualise the timing of the access to the bus (PI1 & PI2 on oscilloscope). Is the deadline met every time?
- c) Correct the application in such a way that the deadline is met every time. Verify using TraceX and the oscilloscope.